

- SQL Básico

¿Qué es una base de datos?

Almacén de información organizada o estructurada en donde sus datos, normalmente poseen una relación o vínculo.

El modelo de las mismas determina su estructura interna, es decir, la manera en que los datos serán almacenados. Por ejemplo:

- Modelo relacional (basado en tablas)
- Modelo dimensional
- Modelo orientado a objetos
- Modelo de grafos

Para poder acceder a la información de una base de datos se emplea un lenguaje de consultas: SQL es el más extendido.

Base de datos relacional

Es un tipo de base de datos que cumple con el modelo relacional y es el modelo de datos más extendido.

Características:

- Los datos son representados como registros o tuplas --> filas y agrupados dentro de relaciones --> tablas.
- La tabla tiene un conjunto de campos o atributos --> columnas
- Las conexiones entre tablas se establecen mediante las claves primarias y claves foráneas y con ellas se garantiza la integridad de la información.

Elementos de una base de datos relacional

● **Relaciones:** permiten almacenar y consultar los datos.

○ **Restricciones:** son limitaciones que se aplican sobre los campos de la base de datos. No son fundamentales ni obligatorios, pero permiten mejorar la organización de los datos.

Dominios: son los tipos de datos que serán almacenados.

Claves:

- ✓ **Primaria:** es la clave única que permite identificar un registro de la tabla.
- ✓ **Foránea:** es la referencia a una clave de otra tabla. No es necesario que sea clave única de la tabla donde está, pero sí de la que está referenciada.
- ✓ **Índice:** permiten un acceso más rápido a los datos y se pueden crear como combinación de los campos de una tabla.

Procedimientos almacenados: código ejecutable que se guarda en la base de datos. Cada motor de base de datos tendrá su lenguaje de programación para los mismos. SQL Server tiene Transact-SQL que permite definir cualquier tarea que se quiera efectuar sobre la base de datos.

Diagrama entidad-relación

Es una herramienta para el modelado de datos que permite representar las entidades relevantes de un sistema de información, sus relaciones y propiedades/atributos.

Elementos de un modelo entidad-relación:

- Entidad --> representa un objeto del mundo real. Pueden ser objetos de existencia física como puede ser una persona o bien con existencia conceptual como lo es una cuenta bancaria.
- Atributo --> son las características que definen las entidades.
- Relación --> establece dependencias entre las entidades. La cardinalidad de una relación indica el número de entidades con las que puede estar relacionada otra entidad.

● **Entidades:** se convierten en tablas y los atributos de la entidad en columnas de la tabla.

○ Relaciones: las relaciones en general se transforman también en tablas en el caso de que la relación tenga atributos.

- Relaciones 1 a 1: cada entidad se convierte en una tabla. La relación es la clave primaria de una de las tablas y foránea en la otra.
- Relaciones 1 a N: cada entidad se convierte en una tabla. Se pasa la clave primaria de la entidad con cardinalidad 1 a la tabla de la otra entidad como clave foránea.
- Relaciones N a M: tanto las entidades como la relación se convierten en tablas. Las claves primarias de las entidades aparecen como clave primaria de la relación.

SQL (Structured Query Language)

- ❖ Lenguaje de acceso a bases de datos relacionales.
- ❖ Basado en el álgebra relacional.
- ❖ No es case-sensitive. Sin embargo, por claridad es importante escribir las sentencias manteniendo un estilo.
- ❖ Cada base de datos comercial cumple con el estándar de SQL aunque existen ligeras diferencias o particularidades entre unos y otras.

Elementos SQL:

Base de datos (DATABASE) es un conjunto de objetos.

Tabla (TABLE) estructura fija de campos.

Campo (COLUMN, FIELD) son las filas de una tabla también denominadas tuplas.

Clave primaria (PK, PRIMARY KEY) identificador unívoco de una fila.

Clave foránea (FK, FOREIGN KEY) establece una relación entre dos tablas.

Tipos de datos

Numéricos --> almacenan enteros o decimales y se dividen en EXACTOS (int, decimal con precisión y escala) y en APROXIMADOS (float, double, real).

Fechas y horas --> DATE almacena solo fechas yyyy-mm-dd, DATETIME combina fecha y hora yyyy-mm-dd hh-mm-ss.

Cadenas --> almacenan cadenas de texto y se clasifican en CHAR que son cadenas de longitud fijas, VARCHAR que son cadenas variables y el parámetro entre paréntesis define la longitud máxima, TEXT/BLOB son para campos de texto más grandes como cuerpos de noticias.

Tipo	Operador	Acción
Aritméticos	+	Operación de suma.
	-	Operación de resta.
	*	Operación de multiplicación.
	/	Operación de división.
	%	Operación de módulo (resto de la división).
Lógicos	AND	TRUE si ambas expresiones booleanas son TRUE.
	OR	TRUE si cualquiera de las dos expresiones booleanas es TRUE.
	NOT	Invierte el valor de cualquier otro operador booleano.

Tipo	Operador	Acción
Comparación	=	Igualdad.
	>	Mayor que...
	>=	Mayor o igual que...
	<	Menor que...
	<=	Menor o igual que...
	IS NULL	Selecciona únicamente los valores nulos (NULL).
	IS NOT NULL	Selecciona todos aquellos valores no nulos (NULL).
	BETWEEN	TRUE si el operando está dentro de un intervalo.
	IN	TRUE si el operando es igual a uno de la lista de expresiones.
	LIKE	TRUE si el operando coincide con un patrón.
	EXISTS	TRUE si una subconsulta contiene cualquiera de las filas.

Consulta SELECT

Se usa para extraer la información de la base de datos. El resultado de la consulta se almacena en otra tabla que se llama result-set.

```
SELECT <column_list>  
FROM <table_list>  
WHERE <conditions>;
```

En donde column_list es la lista de todos los atributos a recuperar por la QUERY y van separados por comas. En el caso de no seleccionar un atributo en particular se emplea el asterisco *. Es posible renombrar una columna o tabla empleando AS. El AS se emplea cuando se realizan cruces de tablas que tengan campos con el mismo nombre así se evitan ambigüedades.

Table_list son la lista de tablas que procesa la QUERY y también van separadas por comas.

Conditions es la expresión booleana que deberán cumplir los registros que devuelva la QUERY, Son opcionales, la consulta no tiene que tener filtros obligatoriamente.

Consulta SELECT - Ejercicio

1. Obtener un listado de todos los empleados
2. Seleccionar únicamente el first_name, last_name y hire_date
3. Renombrar los campos de manera tal que first_name sea nombre, last_name apellido y hire_date sea fecha_contratacion

WHERE clause – Filtrado de datos

Se utiliza dentro de la consulta para filtrar la información devuelta por la consulta. Dentro de la cláusula se pueden emplear los operadores lógicos vistos anteriormente.

```
SELECT <column_list>  
FROM <table_list>  
WHERE <condition> AND OR condition;
```

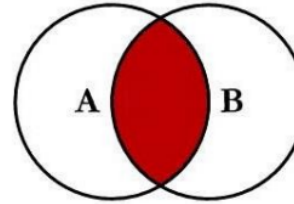
JOIN - Unión de tablas

Las uniones de tablas se emplean para combinar registros o filas de dos o más tablas. Para realizar dicha unión es necesario que las tablas tengan campos en común (PKs y FKs).

INNER JOIN: se comparan las columnas y se encuentran coincidencias entre ambas tablas. Si no se encuentran coincidencias, no se muestra la fila. Solo la intersección se mostrará en los resultados.
Ejemplo: se quiere saber quién es el encargado del departamento de cada área

```
SELECT * FROM dept_manager as DM  
INNER JOIN employees as E  
ON DM.emp_no = E.emp_no;
```

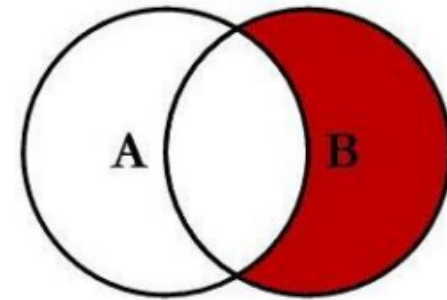
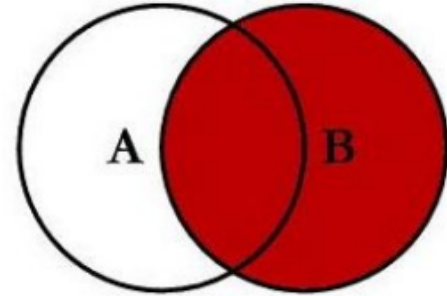
```
SELECT <column_list>  
FROM TableAA  
INNER JOIN TableB B  
ON A.KEY = B.KEY;
```



Devuelve todas las filas de la tabla A con las filas coincidentes de la tabla B. El resultado será NULL en el lado izquierdo cuando no haya ninguna coincidencia. Es posible eliminar todo el lado izquierdo de los resultados.

```
SELECT <column_list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.KEY = B.KEY;
```

```
SELECT <column_list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.KEY = B.KEY  
WHERE B.KEY IS NULL;
```

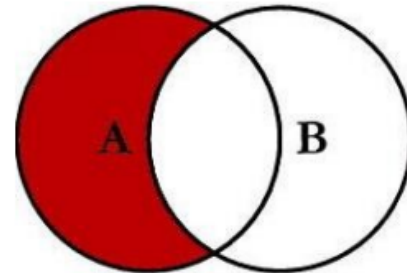
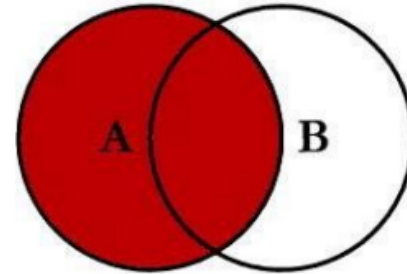


LEFT JOIN - Unión de tablas

Devuelve todas las filas de la tabla A con las filas coincidentes de la tabla B. El resultado será NULL en el lado derecho cuando no haya ninguna coincidencia. Es posible eliminar todo el lado derecho de los resultados.

```
SELECT <column_list>  
FROM TableAA  
LEFT JOIN TableB B  
ON A.KEY = B.KEY;
```

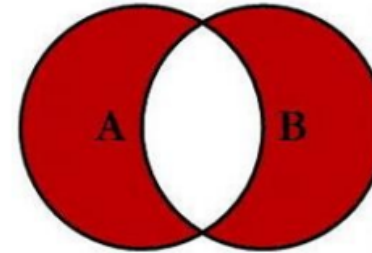
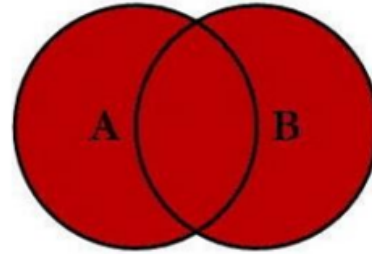
```
SELECT <column_list>  
FROM TableAA  
LEFT JOIN TableB B  
ON A.KEY = B.KEY  
WHERE B.KEY IS NULL;
```



Devuelve todas las filas cuando haya una coincidencia en una de las tablas. El resultado será NULL en el lado izquierdo/derecho cuando no haya ninguna coincidencia.

```
SELECT <column_list>  
FROM TableAA  
FULL OUTER JOIN TableB B  
ON A.KEY = B.KEY;
```

```
SELECT <column_list>  
FROM TableAA  
FULL OUTER JOIN TableB B  
ON A.KEY = B.KEY  
WHERE A.KEY IS NULL  
OR B.KEY IS NULL
```



GROUP BY – Agrupación y agregación

Agrupar los resultados por una o más columnas. Se emplea siempre junto a funciones de agregación

```
SELECT <column_list>, aggregate_function(<column>)  
FROM <table_list>  
[WHERE <conditions>]  
GROUP BY <column_list>;
```

Función	Operación
AVG(column)	Calcula la media de la columna.
COUNT(column)	Cuenta el número de filas.
FIRST(column)	Devuelve el primer valor.
LAST(column)	Devuelve el último valor.
MAX(column)	Devuelve el valor máximo.
MIN(column)	Devuelve el valor mínimo.
SUM(column)	Calcula la suma de todos los registros de la columna.

ORDER BY – Ordenación

Ordena los resultados de una QUERY por una o más columnas del result-set. El orden se puede establecer ascendente o descendente (por default si no se le pone nada, es ascendente). El criterio de ordenación puede ser distinto para cada columna.

```
SELECT <column_list>  
FROM <table_list>  
[WHERE <conditions>]  
ORDER BY <column1> ASCIDESC, <column2> ASCIDESC;
```

UNION – Combinación de resultados

● Permite combinar varios result-set en uno solo. Cada uno de ellos, tiene que tener el mismo número de columnas, ser del mismo tipo y estar en el mismo orden.
○ Si se quieren permitir valores duplicados se debe poner el modificador ALL.

```
SELECT <column_list>
FROM table1
UNION [ALL]
SELECT <column_list>
FROM table2
UNION [ALL]
SELECT <column_list>
FROM table3;
```

LIKE - Patrones

El operador LIKE se emplea sobre una cadena para detectar patrones.

```
SELECT <column_list>  
FROM <table_list>  
WHERE <column> LIKE pattern;
```

Documentación sobre patrones:

<https://learn.microsoft.com/en-us/sql/t-sql/language-elements/like-transact-sql?view=sql-server-ver16>

LIMIT/OFFSET – Paginación

Se emplean para especificar cuántos registros se quieren devolver. No es un estándar, por lo tanto otros lenguajes SQL pueden usar otra sintaxis o bien otros operadores.

```
SELECT <column_list>  
FROM <table_list>  
[WHERE <conditions>]  
LIMIT n OFFSET m;
```

N define el número de registros a devolver

M determina la posición inicial (produciendo un desplazamiento desde el primer registro)

CREATE – Creación de base de datos

Para crear una base de datos vacía:

```
CREATE DATABASE <database_name>;
```

DROP – Eliminación de base de datos

```
DROP DATABASE <database_name>;
```

USE – Utilización de base de datos

```
USE <database_name>;
```

```
CREATE TABLE <table_name>
(  
  <column_name1> data_type1,  
  <column_name2> data_type2,  
  <column_name3> data_type3,  
  ...  
);
```

table_name es el nombre de la nueva table a crear

column_name y data_type son las columnas de la table con el tipo de datos de cada una de ellas.

Las restricciones (CONSTRAINTS) sirven para establecer reglas y comprobaciones sobre los datos. Una de las más comunes es la clave primaria.

PRIMARY KEY

Es posible añadirle una clave primaria a una tabla en el momento de la creación. La PK debe tener valores únicos y no nulos.

```
CREATE TABLE <table_name>
(
  <column_name> data_type,
  ...,
  [CONSTRAINT <pk_name>] PRIMARY KEY (<column_name>,...)
)
```

pk_name es el nombre de la clave primaria a crear.

Column_name es la columna que formará parte de la clave primaria de la tabla.

Restricciones – Foreign Key

Tal como en el caso anterior, es posible agregar una clave foránea a una tabla en el momento de creación de la misma.

```
CREATE TABLE <table_name>
(
  <column_name> data_type,
  ...,
  [CONSTRAINT <fk_name>] FOREIGN KEY (<column_name>)
  REFERENCES <table_name_ref>(<pk_table_name_ref>)
);
```

fk_name es el nombre de la clave a crear.

column_name es la columna que tendrá asociada la restricción

table_name_ref es el nombre de la table que hace referencia a la clave foránea

pk_table_name_ref es el campo que hace referencia a la clave foránea

DEFAULT constraint

Permite establecer un valor por defecto para una columna en el caso de que se deje vacía al momento de la inserción.

```
CREATE TABLE <table_name>
(
  <column_name> data_type DEFAULT <default_value>,
  ...,
);
```

El default_value es el valor por defecto de la columna y tiene que coincidir su tipo de datos con el que almacena la columna.

INSERT - Inserciones

Se utiliza para insertar nuevos registros en una tabla de la base de datos

```
INSERT INTO <table_name>[(<column_list>)]  
VALUES (<value_list>);
```

Table_name es la tabla donde se insertará el nuevo registro

Column_list es la lista de las columnas o campos a rellenar. Si no se establecen, la base de datos entiende que se pasan todos los campos de la tabla en el orden que fueron creados.

Value_list son los valores a insertar en el nuevo registro de la tabla.

UPDATE - Actualizaciones

Se utiliza para actualizar registros existentes en una tabla de la base de datos

```
UPDATE <table_name>  
SET <column1> = <value1>, <column2> = <value2>  
[WHERE <conditions>];
```

Table_name es la tabla donde se actualizará el registro

Column1 = value1 son los nuevos valores para las columnas que sea necesario actualizar.

Conditions se utilizan cuando se quieren actualizar solo los registros que cumplan cierta condición. Si no se establecen condiciones, se actualizan TODOS los registros de la tabla.

DELETE - Borrado

Se utiliza para eliminar registros existentes en una tabla de la base de datos

```
DELETE  
FROM <table_name>  
[WHERE <conditions>]
```

Table_name es la tabla a eliminar

Conditions se utilizan cuando se quiere borrar solo los registros que cumplan cierta condición. Si no se establecen condiciones, se borran TODOS los registros de la tabla.

Si se quiere borrar el contenido completo de una tabla se emplea la sentencia TRUNCATE en vez de DELETE sin clausula where.

```
TRUNCATE TABLE <table_name>;
```